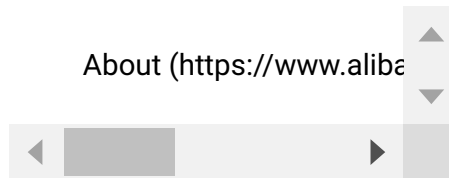


(https://account.alibabacloud.com/login/login.htm)


[International - English](#)
[Cart \(https://cart.alibabacloud.com/\)](#)

[Contact Sales \(https://www.alibabacloud.com/contact-sales\)](#)
[Topic Center](#)
[Console \(https://www.alibabacloud.com\)](#)
[Log In \(https://account.alibabacloud.com/login/login.htm?\)](#)
[oauth_callback=https%3A%2F%2Ftopic.alibabacloud.com%2Fa%2Fsummary-of-process-context-and-interrupt-context_8_8_30142056.html\)](#)
[Sign Up \(https://account.alibabacloud.com/register/intl_register.htm?\)](#)
[oauth_callback=https%3A%2F%2Ftopic.alibabacloud.com%2Fa%2Fsummary-of-process-context-and-interrupt-context_8_8_30142056.html\)](#)
[About \(https://www.aliba](#)

[Home \(/topic.alibabacloud.com/\)](#) > [Others \(/topic.alibabacloud.com/c/others_8_1\)](#)

Summary of process context and interrupt context

Last Update:2018-02-23

Source: Internet

Author: User

Developer on Alibaba Coud: Build your first app with APIs, SDKs, and tutorials on the Alibaba Cloud. Read more >
<https://developer.alibabacloud.com>

1. Preface

Recent knowledge of the Linux kernel often sees user space and kernel space and process context and interrupt context. Look very familiar, half-day and can not say what is going on, what is the difference. The process of reading is often felt deceived, indefinitely feeling, very uncomfortable, and today a good combination of books and online information to summarize, deepen understanding.

2. User space and kernel space

We know that the operating system is now using virtual memory, then for 32-bit operating systems, its addressing space (virtual storage space) is 4G (2 of 32). The core of the worry system is the kernel, which is independent of the normal application, has access to the protected memory space, and has all the permissions to access the underlying hardware device. In order to ensure that the user process can not directly manipulate the kernel, to ensure the security of the kernel, worry about the system to divide the virtual space into two parts, part of the kernel space, part of the user space. For the Linux operating system, the highest 1G bytes (from the virtual address 0xc0000000 to 0xffffffff) for the kernel to use, called the kernel space, and the lower 3G bytes (from the virtual address 0x00000000 to 0xbfffffff) for each process to use, Called User space. Each process can enter the kernel through system calls, so the Linux kernel is shared by all processes within the system. Thus, from a specific process perspective, each process can have a virtual space of 4G bytes. The spatial allocation is as follows:



with user space and kernel space, the entire Linux internal structure can be divided into three parts, from the bottom to the top, in order: hardware----- as shown in the following:

Detailed Questions to note:

(1) kernel space is stored in the kernel code and data, and the process of user space is stored in the user program code and data. **both the kernel space and the user space are in virtual space .**

(2) Linux uses a level two protection mechanism: level 0 for the kernel, and 3 for user programs.

Kernel state and User state:

(1) When **a task (process) executes a system call and is executed in the kernel code, the process is said to be in the Kernel runtime State (kernel state).** at this point the processor is executed in the highest privileged (level 0) kernel code. When the process is in the kernel state, the kernel code that executes will use the kernel stack of the current process. Each process has its own kernel stack.

(2) **when the process executes the user's own code, it is said to be in the user's running state (user state).** At this point the processor runs in the least privileged (level 3) user code. When the user program is being executed and the program is interrupted abruptly, the user program can also be symbolically referred to as the kernel state in the process. Because the interrupt handler will use the kernel stack of the current process.

3. Process context and interrupt context

I'm looking at the third chapter of the book, "Linux kernel design and implementation". Process management, see the process context. The book says that when a program executes a system call or triggers an exception (soft interrupt), it falls into kernel space, which at this point represents the process execution and is in the context of the process. Look after still did not figure out, what is the process context, how on Google above a crazy search, summed up as follows:

The program in the execution process usually has the user state and the kernel state two states, the CPU is in the kernel state according to the context Environment further subdivision, therefore has the following three kinds of states:

- (1) kernel state, running in the process context, the kernel represents the process running in the kernel space.
- (2) The kernel state, running in the interrupt context, the kernel represents the hardware running in the kernel space.
- (3) User mode, running in user space.

Context: Context is simply an environment.

User-space applications, through system calls, into the kernel space. At this time, the process of user space to pass a lot of variables, parameters of the value to the kernel, the kernel state when running the user process to save some register values, variables and so on. The **so-called "process context" can be seen as the parameters that the user process passes to the kernel and the set of variables and register values that the kernel is going to save and the environment at that time.**

In relation to a process, it is the environment at which the process executes. In particular, variables and data, including all register variables, process open files, memory information, and so on. The context of a process can be divided into three parts: User-level context, register context, and system-level contexts.

- (1) User-level context: body, data, user stack, and shared storage area;
- (2) Register context: Universal Register, Program Register (IP), processor status register (eflags), stack pointer (ESP);

(3) System level context: Process Control block task_struct, memory management information (mm_struct, Vm_area_struct, PGD, Pte), kernel stack.

when a process is scheduled, a process switch is a context switch. The operating system must switch all the information mentioned above to run the newly scheduled process. The system calls the mode switch. Mode switching is much easier compared to process switching, and saves time because the main task of mode switching is to switch the process register context.

The hardware passes the trigger signal, causing the kernel to call the interrupt handler and enter the kernel space. In this process, some variables and parameters of the hardware are also passed to the kernel, and the kernel is interrupted by these parameters. The **so-called "interrupt context" can actually be seen as the hardware passes over the parameters and some other environment that the kernel needs to save (mainly the process environment that is currently interrupted)**. when interrupted, the kernel does not run on behalf of any process, it generally accesses only the system space and does not access the process space, and the kernel generally does not block when executed in an interrupt context.

The contents of the excerpt from Linux comments are as follows:

Process Context

One of the most important parts of a process are the executing program code. This code was read in from a executable file and executed within the program's address space. Normal program execution occurs in User-space. When a program executes a system call or triggers an exception, it enters Kernel-space. At this point, the kernel are said to being "executing on behalf of the process" and are in process context. When in process context, the current macro is valid[7]. Upon exiting the kernel, the process resumes execution in User-space, unless a higher-priority process have become runnable In the interim (transition period), in which case the scheduler is invoked to select the higher priority process.

Other than process context there was interrupt context, in interrupt context, the system was not running on behalf of a process, but was executing an interrupt handler. There is no process tied to interrupt handlers and consequently no process context.

System calls and exception handlers is well-defined interfaces into the kernel. A process can begin executing in kernel-space only through one of these interfaces – all access to the kernel is through These interfaces.

Interrupt Context

When executing a interrupt handler or bottom half, the kernel is in interrupt context. Recall That process context is the mode of operation the kernel are in while it's executing on behalf of a process-- For example, executing a system call or running a kernel thread. In process context, the current macro points to the associated task. Furthermore, because a process is coupled to the kernel in process context (because the process is connected to the kernel in the same way as the process above), process context can Sleep or otherwise invoke the scheduler.

Interrupt context, on the other hand, was not associated with a process. The current macro isn't relevant (although it points to the interrupted process). Without a backing process (because there is no process background), interrupt context cannot sleep-how would it ever reschedule? (or how to reschedule it again?) Therefore, cannot call certain functions from interrupt context. If A function sleeps, you cannot use it from your interrupt handler--this limits the functions so one can call from an Interrupt handler. (This is the limit on what functions can be used in an interrupt handler)



Interrupt context is time critical because the Interrupt handler interrupts other code. Code should be quick and simple. Busy looping is discouraged. This was a very important point; Always keep in mind that your interrupt handler has interrupted other code (possibly even another interrupt handler on a different line!). Because of this asynchronous nature, it is imperative (must) that all interrupt handlers are as quick and as simple as possible. As much as possible, work should be pushed out from the interrupt handler and performed in a bottom half, which runs at a more convenient time.

The setup of an interrupt handler's stacks is a configuration option. Historically, interrupt handlers did not receive (own) their own stacks. Instead, they would share the stack of the process that they interrupted[1]. The kernel stack is a fixed size; Typically, that's 8KB on 32-bit architectures and 16KB on 64-bit architectures. Because in this setup interrupt handlers share the stack, they must be exceptionally frugal (must be very economical) with what data they allocate there. Of course, the kernel stack is limited to begin with, so all kernel code should be cautious.

A process is always running. When nothing else is schedulable, the idle task runs.

A paragraph in the Linux full comment:

When a process executes, the values in all registers of the CPU, the state of the process, and the contents of the stack are referred to as the context of the process. When the kernel needs to switch to another process, it needs to save all the state of the current process, that is, to save the context of the current process so that it can be executed when the process is executed again. In Linux, the current process context is stored in the task data structure of the process. In the event of an outage, the kernel executes the interrupt service routine under the kernel state in the context of the interrupted process. At the same time, all resources that need to be used are preserved so that the execution of the interrupted process can be resumed at the end of the relay service.

Linux kernel space and user space detailedLinux drivers generally work in kernel space, but can also work in user space. Below we will analyze in detail what is kernel space, what is user space, and how to judge them.

Linux simplifies the segmentation mechanism so that the virtual address is always consistent with the linear address, so the virtual address space for Linux is also 0~4G. The Linux kernel divides this 4G-byte space into two parts. The highest 1G bytes (from virtual addresses 0xc0000000 to 0xffffffff) are used by the kernel, which is called "kernel space". Instead, the lower 3G bytes (from the virtual address 0x00000000 to 0xbfffffff) are used by each process, called "User space." Because each process can enter the kernel through system calls, the Linux kernel is shared by all processes within the system. Thus, from a specific process perspective, each process can have a virtual space of 4G bytes.

Linux uses level two protection: level 0 for kernel use, and 3 for user programs. As you can see (it is not possible to represent a graph), each process has its own private user space (0~3G), which is invisible to other processes in the system. The highest 1GB bytes virtual kernel space is shared by all processes and the kernel.

Kernel space is the kernel code and data, while the process of user space is stored in the user program code and data. Both the kernel space and the user space are in virtual space.

Although kernel space occupies up to 1GB bytes in each virtual space, mapping to physical memory always starts with the lowest address (0x00000000). For kernel space, the address map is a very simple linear mapping, 0xc0000000 is the physical address and linear address between the amount of displacement, in the Linux code is called Page_offset.



How is the communication between kernel space and user space?

Kernel space and user space are generally communicated through system calls.

How can I tell if a driver is a user-mode driver or a kernel-mode driver? What are the criteria for judging?

The driver of the user-space mode typically accomplishes the hardware access through system calls, such as mapping the drive IO space to user space through system calls. Therefore, the main basis of judgment is the system call.

Kernel space and user space are different too much, say no, such as user-state linked list and kernel chain list is not the same, the user state with printf, the kernel state with PRINTK; the user state each application space is virtual, relatively independent, the kernel state is not independent, so programming to be very careful. Wait a minute.

There are many methods of user-state and kernel-state program communication, not only the system call, in fact, the system call is a bad choice, because the need for system call number, this needs to be unified distribution.

Can be done by IOCTL, SYSFS, Proc and so on.

Kernel State and User state

When a task (process) executes a system call and is executed in the kernel code, we say that the process is in the kernel run state (or simply the kernel state). At this point the processor is executed in the highest privileged (level 0) kernel code. When the process is in the kernel state, the kernel code that executes will use the kernel stack of the current process. Each process has its own kernel stack. When the process executes the user's own code, it is said to be in the user's running state (user state). That is, the processor is running in the least privileged (level 3) user code. When the user program is being executed and the program is interrupted abruptly, the user program can also be symbolically referred to as the kernel state in the process. Because the interrupt handler will use the kernel stack of the current process. This is somewhat similar to the state of a process that is in a kernel state.

process context and interrupt context The processor is always in one of the following states:

- 1, the kernel state, running in the process context, the kernel represents the process running in the kernel space;
- 2, the kernel state, running in the interrupt context, the kernel represents the hardware running in the kernel space;
- 3, user state, running in user space.

User-space applications, through system calls, into the kernel space. At this time, the process of user space to pass a lot of variables, parameters of the value to the kernel, the kernel state when running the user process to save some register values, variables and so on. The so-called "process context" can be seen as the parameters that the user process passes to the kernel and the set of variables and register values that the kernel is going to save and the environment at that time.

The hardware passes the trigger signal, causing the kernel to call the interrupt handler and enter the kernel space. In this process, some variables and parameters of the hardware are also passed to the kernel, and the kernel is interrupted by these parameters. The so-called "interrupt context" can actually be seen as the hardware passes over the parameters and some other environment that the kernel needs to save (mainly the process environment that is currently interrupted). kernel space and user space



The first few sections are mainly about the external architecture of Linux, in this section, will analyze the internal structure of Linux, initially can be divided into three layers of the internal system: Hardware = Kernel Space + User Space

1. Why divide into kernel space and user space?

Linux kernel is the core of the operating system, independent of ordinary applications, access to protected memory space, and access to all the permissions of the underlying hardware device.

Linux kernel is the core of the operating system, independent of ordinary applications, access to protected memory space, and access to all the permissions of the underlying hardware device. for kernel Such a high security level, it is obvious that other applications are not allowed to be arbitrarily invoked or accessed, so there is a need for kernel to provide a certain protection mechanism, the protection mechanism used to tell those applications, you can only access certain licensed resources, Unauthorized resources are denied access, so the kernel and the upper-level applications are isolated, respectively, called kernel space and user Space.

2. How does the user space program access the kernel space?

The above mentioned that the user state and the kernel state is two isolated space, although logically extracted from the isolation, but inevitably, there will always be some user space to access the kernel space resources, how to do?

The above mentioned that the user state and the kernel state is two isolated space, although logically extracted from the isolation, but inevitably, there will always be some user space to access the kernel space resources, how to do?

as can be seen from the structure, the Kernel space layer from bottom to top includes:

Arch: corresponds to the kernel arch directory, which contains support for such architectures as x86, IA64, ARM, s390, etc.

Device Driver: Corresponding to the kernel drivers directory, including block, char, NET, USB and other hardware-driven support;

Above arch and driver, it is the support of memory, process, file system, network protocol stack and so on.

the top layer is the system call Interface, called interface, as its name, this layer is the user space and kernel space Bridge, the user space application through the system call this unified portal to access the hardware resources in the systems, through this interface, All resource access is performed under the control of the kernel, so as to avoid unauthorized access to the system resources by the user program, thus guaranteeing the security and stability of the system.

3. Role of the GLIBC library

In the user space layer, you can see the existence of the GLIBC library, the reason why it is listed as a separate emphasis, because the general user-space program does not directly invoke the kernel system call to access systems resources, but by glibc such a library indirectly called system call, in other words, GLIBC has a layer of encapsulation of kernel's system call.

4. C Library of kernel and C Library of GLIBC

Previously emphasized, kernel is relatively independent of the existence, do not rely on any user space of the program or library, and we all know that kernel in addition to a small number of assembly code, most of them are written in C language, glibc library belongs to user space, no glibc library support, Kernel How to deal with C language-related code?

Summary of process context and interrupt context



This article is an English version of an article which is originally in the Chinese language on aliyun.com and is provided for information purposes only. This website makes no representation or warranty of any kind, either expressed or implied, as to the accuracy, completeness ownership or reliability of the article or any translations